

数值分析期末上机报告

学号：241098117 姓名：张城宗

1 题目一：单步法与 Adams PECE 方法比较

1.1 问题描述

求解初值问题

$$y' = -\frac{1}{x^2} - \frac{y}{x} - y^2, \quad 1 \leq x \leq 2, \quad y(1) = -1,$$

取固定步长 $h = 0.1$ 。

1. 分别使用向前 Euler、改进 Euler、Heun、中点和经典四阶 Runge-Kutta 方法计算数值解，并比较不同方法的结果。
2. 使用 RK4 提供起步值，实现四阶 Adams PECE 预测校正方法，并与 RK4 比较。

1.2 方法介绍

向前 Euler 方法采用当前节点斜率推进：

$$y_{n+1} = y_n + hf(x_n, y_n),$$

它实现简单，但只有一阶精度。

改进 Euler、Heun 和中点方法均属于二阶显式 Runge-Kutta 方法。以改进 Euler 为例，

$$K_1 = f(x_n, y_n), \quad K_2 = f(x_n + h, y_n + hK_1), \quad y_{n+1} = y_n + \frac{h}{2}(K_1 + K_2).$$

经典 RK4 方法为

$$\begin{aligned} K_1 &= f(x_n, y_n), & K_2 &= f(x_n + h/2, y_n + hK_1/2), \\ K_3 &= f(x_n + h/2, y_n + hK_2/2), & K_4 &= f(x_n + h, y_n + hK_3), \\ y_{n+1} &= y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4). \end{aligned}$$

它每步需要四次函数计算，理论收敛阶为四阶。

四阶 Adams PECE 方法先用四步 Adams-Bashforth 公式预报，

$$y_{n+1}^P = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}),$$

再用 Adams-Moulton 公式校正，

$$y_{n+1} = y_n + \frac{h}{24}(9f(x_{n+1}, y_{n+1}^P) + 19f_n - 5f_{n-1} + f_{n-2}).$$

由于多步法需要历史值，前三步使用 RK4 起步。

1.3 数值结果

表 1: 不同方法在终点 $x = 2$ 处的数值解

方法	终点数值解 $y(2)$
向前 Euler	-3.5510956192
改进 Euler	-5.1032101606
Heun	-5.0041974679
中点	-4.9557885785
RK4	-5.4009915651
Adams PECE	-5.3802222860

Adams PECE 与 RK4 在终点的绝对差为

$$|y_{\text{PECE}}(2) - y_{\text{RK4}}(2)| \approx 2.077 \times 10^{-2}.$$

1.4 结果分析

1. 本题解在区间后半段下降较快，因此不同格式在步长 $h = 0.1$ 下产生了较明显差异。向前 Euler 只有一阶精度，与高阶方法结果差异最大。
2. 三种二阶方法的结果较为接近，但仍与 RK4 存在明显差异，说明该步长对于本题并不算小。
3. Adams PECE 使用 RK4 起步，其结果与 RK4 最接近。它的主循环中复用历史函数值，每步函数计算次数少于 RK4，适合固定步长下的连续计算。
4. 原数值 ODE 报告中本题终点结果约为 -0.6 ，但该结果与报告所写方程不一致；本报告中的数据由附录 Python 程序按上述方程重新计算得到。

2 题目二：二阶方程化为一阶系统并验证 RK4 收敛阶

2.1 问题描述

求解二阶常微分方程初值问题

$$y'' + \frac{2}{x}y' - \frac{6}{x^2}y = 5 - 6x + 7x^2, \quad 1 < x < 2,$$

$$y(1) = \frac{1}{2}, \quad y'(1) = 2.$$

精确解为

$$y(x) = x^2 - x^3 + \frac{1}{2}x^4 + x^2 \ln x.$$

分别取步长

$$h = \frac{1}{200}, \frac{1}{100}, \frac{1}{50}, \frac{1}{25}, \frac{1}{10}, \frac{1}{5}, \frac{1}{4},$$

使用经典 RK4 方法计算最大绝对误差并分析收敛阶。

2.2 方法介绍

令

$$w_1 = y, \quad w_2 = y',$$

则二阶方程可化为一阶系统

$$\mathbf{w}' = \begin{bmatrix} w_2 \\ -\frac{2}{x}w_2 + \frac{6}{x^2}w_1 + 5 - 6x + 7x^2 \end{bmatrix}, \quad \mathbf{w}(1) = \begin{bmatrix} 1/2 \\ 2 \end{bmatrix}.$$

对向量状态 $\mathbf{w}$ 使用 RK4 时，计算公式与标量情形相同。第一分量 w_1 即为原方程的数值解。

定义最大绝对误差

$$E(h) = \max_n |w_{1,n} - y(x_n)|.$$

对相邻两个步长，数值收敛阶估计为

$$p \approx \frac{\log(E(h_1)/E(h_2))}{\log(h_1/h_2)}.$$

2.3 数值结果

表 2: 不同步长下 RK4 最大误差与估计收敛阶

步长 h	最大绝对误差 $E(h)$	估计收敛阶 p
1/200	2.4661×10^{-10}	—
1/100	3.9227×10^{-9}	3.9915
1/50	6.1913×10^{-8}	3.9803
1/25	9.6405×10^{-7}	3.9608
1/10	3.4751×10^{-5}	3.9123
1/5	4.8875×10^{-4}	3.8140
1/4	1.1220×10^{-3}	3.7243

2.4 结果分析

1. 当步长由 $1/4$ 缩小至 $1/200$ 时，最大误差由约 10^{-3} 降至约 10^{-10} ，表明 RK4 对该光滑问题具有较高精度。
2. 在较小步长下，估计收敛阶接近 4，与经典 RK4 的理论四阶精度一致。较大步长下估计阶略低，是因为渐近误差关系 $E(h) \approx Ch^4$ 尚未完全成立。
3. 将高阶方程化为一阶系统后，可以直接复用向量 RK4 程序；这种状态变量方法也适用于更高阶方程和一般常微分方程组。

2.5 结论

两道题均使用 Python 完成数值计算。第一题说明不同单步法和多步法在精度与计算量上存在差异；第二题验证了经典 RK4 对光滑问题具有四阶收敛性。实际计算时应根据精度要求、稳定性以及函数计算成本选择方法和步长。

附录: Python 程序

A.1 题目一 Python 程序

Listing 1: 题目一: 五种单步法与 Adams PECE 比较

```

1 import numpy as np # 导入 NumPy; 用于创建网格、预分配数组和进行向量运算。
2 import matplotlib.pyplot as plt # 导入 Matplotlib; 用于绘制数值解比较图。
3
4 f = lambda x, y: -1 / x**2 - y / x - y**2 # 定义 ODE 右端 f(x,y); ** 表示乘方。
5 x0, x_end, y0, h = 1.0, 2.0, -1.0, 0.1 # 设置区间、初值和固定步长。
6 N = round((x_end - x0) / h) # 计算固定步长下的总推进步数。
7 x = x0 + np.arange(N + 1) * h # 构造等距节点 x_0,...,x_N。
8
9 def rk4(f, x, y0, h): # 定义经典四阶 Runge--Kutta 求解函数。
10     y = np.zeros(x.size) # 预分配数值解数组, 避免循环中反复扩容。
11     y[0] = y0 # 写入初值 y(x_0)=y_0。
12     for n in range(x.size - 1): # 从左端点逐步推进到右端点。
13         k1 = f(x[n], y[n]) # 第一阶段使用当前点斜率。
14         k2 = f(x[n] + h/2, y[n] + h*k1/2) # 第二阶段在预测中点计算斜率。
15         k3 = f(x[n] + h/2, y[n] + h*k2/2) # 第三阶段再次修正中点斜率。
16         k4 = f(x[n] + h, y[n] + h*k3) # 第四阶段在右端预测点计算斜率。
17         y[n+1] = y[n] + h*(k1 + 2*k2 + 2*k3 + k4)/6 # 加权组合四个阶段, 得到四阶近似。
18     return y # 返回全部节点上的 RK4 数值解。
19
20 y_euler = np.zeros(N + 1) # 预分配向前 Euler 数值解。
21 y_improved = np.zeros(N + 1) # 预分配改进 Euler 数值解。
22 y_heun = np.zeros(N + 1) # 预分配二阶 Heun 数值解。
23 y_midpoint = np.zeros(N + 1) # 预分配中点方法数值解。
24 for y in (y_euler, y_improved, y_heun, y_midpoint): # 依次访问四个数值解数组。
25     y[0] = y0 # 为每种单步法写入相同初值。
26
27 for n in range(N): # 对四种单步法同时逐步推进。
28     y_euler[n+1] = y_euler[n] + h*f(x[n], y_euler[n]) # 向前 Euler 是一阶显式方法。
29     k1 = f(x[n], y_improved[n]) # 改进 Euler 的左端斜率。
30     k2 = f(x[n] + h, y_improved[n] + h*k1) # 在 Euler 预测点计算右端斜率。
31     y_improved[n+1] = y_improved[n] + h*(k1+k2)/2 # 两端斜率平均得到二阶结果。
32     k1 = f(x[n], y_heun[n]) # Heun 方法的第一阶段斜率。
33     k2 = f(x[n] + 2*h/3, y_heun[n] + 2*h*k1/3) # 在 2/3 步位置计算第二阶段。
34     y_heun[n+1] = y_heun[n] + h*(k1+3*k2)/4 # 使用满足二阶条件的权重组合。
35     k1 = f(x[n], y_midpoint[n]) # 中点方法先计算当前点斜率。
36     k2 = f(x[n] + h/2, y_midpoint[n] + h*k1/2) # 用 k1 预测中点并计算中点斜率。
37     y_midpoint[n+1] = y_midpoint[n] + h*k2 # 使用中点斜率推进整步。
38
39 y_rk4 = rk4(f, x, y0, h) # 调用 RK4 函数计算四阶参考数值解。
40 y_pece = np.zeros(N + 1) # 预分配 Adams PECE 数值解。
41 f_values = np.zeros(N + 1) # 保存历史函数值, 以供多步公式重复使用。
42 y_pece[:4] = y_rk4[:4] # 使用 RK4 提供 y_0,y_1,y_2,y_3 四个起步值。
43 for i in range(4): # 计算四个起步节点对应的函数值。
44     f_values[i] = f(x[i], y_pece[i]) # 保存 f_i=f(x_i,y_i)。
45 for n in range(3, N): # 从已有四个历史点的位置开始执行 PECE。

```

```

46     y_pred = y_pece[n] + h*(55*f_values[n]-59*f_values[n-1]+37*f_values[n-2]-9*f_values[n-3])/24 # AB4 预报。
47     f_pred = f(x[n+1], y_pred) # 在预测值处计算新节点函数值。
48     y_pece[n+1] = y_pece[n] + h*(9*f_pred+19*f_values[n]-5*f_values[n-1]+f_values[n-2])/24 # AM4 校正。
49     f_values[n+1] = f(x[n+1], y_pece[n+1]) # 保存校正值对应的函数值, 供下一步使用。
50
51 print("method          y(2)") # 打印结果表表头。
52 for name, y in [("Euler", y_euler), ("Improved Euler", y_improved), ("Heun", y_heun), ("Midpoint", y_midpoint), ("RK4", y_rk4), ("Adams PECE", y_pece)]: # 配对方法名和结果数组。
53     print(f"{name:15s} {y[-1]:.10f}") # 输出每种方法的终点数值解。
54 print("PECE-RK4 difference =", abs(y_pece[-1] - y_rk4[-1])) # 输出 PECE 与 RK4 的终点差。
55
56 for name, y in [("Euler", y_euler), ("Improved Euler", y_improved), ("Heun", y_heun), ("Midpoint", y_midpoint), ("RK4", y_rk4), ("Adams PECE", y_pece)]: # 遍历全部方法。
57     plt.plot(x, y, marker="o", label=name) # 在同一坐标系绘制各方法数值解。
58 plt.xlabel("x") # 设置横轴名称。
59 plt.ylabel("y") # 设置纵轴名称。
60 plt.grid(True) # 打开网格, 便于比较曲线。
61 plt.legend() # 显示方法图例。
62 plt.tight_layout() # 自动调整边距, 避免文字被裁切。
63 plt.savefig("python_pi_solutions.pdf") # 保存矢量 PDF, 供 LaTeX 自动插入。
64 plt.close() # 关闭当前图, 释放绘图资源。

```

A.2 题目二 Python 程序

Listing 2: 题目二: 二阶 ODE 化为一阶系统并验证 RK4 收敛阶

```

1 import numpy as np # 导入 NumPy; 用于向量状态、网格、误差和对数运算。
2 import matplotlib.pyplot as plt # 导入 Matplotlib; 用于绘制解曲线和双对数误差图。
3
4 def F(x, w): # 定义一阶系统右端; w[0]=y, w[1]=y'。
5     return np.array([w[1], -2*w[1]/x + 6*w[0]/x**2 + 5 - 6*x + 7*x**2]) # 返回列状态的两个导数分量。
6
7 def exact(x): # 定义题目给出的精确解。
8     return x**2 - x**3 + 0.5*x**4 + x**2*np.log(x) # NumPy 运算自动逐元素作用于数组。
9
10 def solve_rk4(h): # 定义给定步长下的向量 RK4 求解器。
11     x0, x_end = 1.0, 2.0 # 设置积分区间。
12     N = round((x_end - x0) / h) # 计算总步数。
13     x = x0 + np.arange(N + 1) * h # 创建等距节点。
14     w = np.zeros((N + 1, 2)) # 每一行保存一个节点的 [y, y']。
15     w[0] = [0.5, 2.0] # 写入初始状态 [y(1), y'(1)]。
16     for n in range(N): # 对向量状态逐步执行 RK4。
17         k1 = F(x[n], w[n]) # 第一阶段斜率向量。
18         k2 = F(x[n] + h/2, w[n] + h*k1/2) # 第二阶段中点斜率向量。
19         k3 = F(x[n] + h/2, w[n] + h*k2/2) # 第三阶段修正的中点斜率向量。

```

```
20     k4 = F(x[n] + h, w[n] + h*k3) # 第四阶段右端斜率向量。
21     w[n+1] = w[n] + h*(k1 + 2*k2 + 2*k3 + k4)/6 # 使用 RK4 权重更新全部状态分量。
22     return x, w # 返回节点与完整状态数组。
23
24 h_values = np.array([1/200, 1/100, 1/50, 1/25, 1/10, 1/5, 1/4]) # 设置题目要求的步长。
25 errors = np.zeros(h_values.size) # 预分配每个步长对应的最大绝对误差。
26 for i, h in enumerate(h_values): # enumerate 同时给出数组下标和步长。
27     x, w = solve_rk4(h) # 计算当前步长下的向量数值解。
28     errors[i] = np.max(np.abs(w[:, 0] - exact(x))) # 第一分量是 y; 对全部节点误差取最大值。
29
30 orders = np.full(h_values.size, np.nan) # 第一组数据没有前一误差, 因此初始填为 nan。
31 for i in range(1, h_values.size): # 从第二个步长开始估计相邻误差的收敛阶。
32     orders[i] = np.log(errors[i-1]/errors[i]) / np.log(h_values[i-1]/h_values[i]) # 使用
    一般步长比公式。
33
34 print("h          max_error          order") # 打印误差与阶数表头。
35 for h, error, order in zip(h_values, errors, orders): # zip 将三组对应数据逐行配对。
36     print(f"{h:.6f}    {error:.10e}    {order:.6f}") # 格式化输出步长、最大误差与估计阶。
37
38 x, w = solve_rk4(1/200) # 使用最小步长生成数值解与精确解对比图。
39 plt.plot(x, exact(x), "k-", label="exact") # 绘制精确解实线。
40 plt.plot(x, w[:, 0], "r--", label="RK4") # 绘制 RK4 第一分量虚线。
41 plt.xlabel("x") # 设置横轴名称。
42 plt.ylabel("y") # 设置纵轴名称。
43 plt.grid(True) # 打开网格。
44 plt.legend() # 显示精确解与数值解图例。
45 plt.tight_layout() # 自动调整图像边距。
46 plt.savefig("python_p2_solution.pdf") # 保存解曲线图供 LaTeX 插入。
47 plt.close() # 关闭解曲线图。
48
49 plt.loglog(h_values, errors, "bo-", label="RK4 error") # 在双对数坐标绘制误差曲线。
50 reference = errors[0] * (h_values / h_values[0])**4 # 构造经过首个误差点的四阶参考线。
51 plt.loglog(h_values, reference, "k--", label="O(h^4)") # 绘制理论四阶误差参考线。
52 plt.xlabel("step size h") # 设置横轴为步长。
53 plt.ylabel("maximum error") # 设置纵轴为最大绝对误差。
54 plt.grid(True, which="both") # 对主刻度和次刻度均显示网格。
55 plt.legend() # 显示误差曲线和参考线图例。
56 plt.tight_layout() # 自动调整图像边距。
57 plt.savefig("python_p2_convergence.pdf") # 保存收敛阶图供 LaTeX 插入。
58 plt.close() # 关闭误差图并释放资源。
```